Podstawy Uczenia Maszynowego

Czym się zajmiemy na ćwiczeniach?

~~Ćwiczenia 1: metody liniowe~~

~~Ćwiczenia 2: metody oparte na sąsiedztwie~~

Ćwiczenia 3: metody jądrowe

Ćwiczenia 4: metody probabilistyczne

Ćwiczenia 5: klasyfikacja niezbalansowana

Ćwiczenia 6: selekcja cech i redukcja wymiarowości

Ćwiczenia 7: klasteryzacja

Metody jądrowe (kernel methods)

Źródła

Filmy:

- Cornell CS 5787: Applied Machine Learning, wykłady 9-11
- MIT: Artificial Intelligence, wykład "Support Vector Machines"
- P. Jahangiry: Machine Learning Concepts, wykłady 23-27
- Udemy: Machine Learning and AI: Support Vector Machines in Python

Źródła

Tutoriale / prezentacje / artykuły:

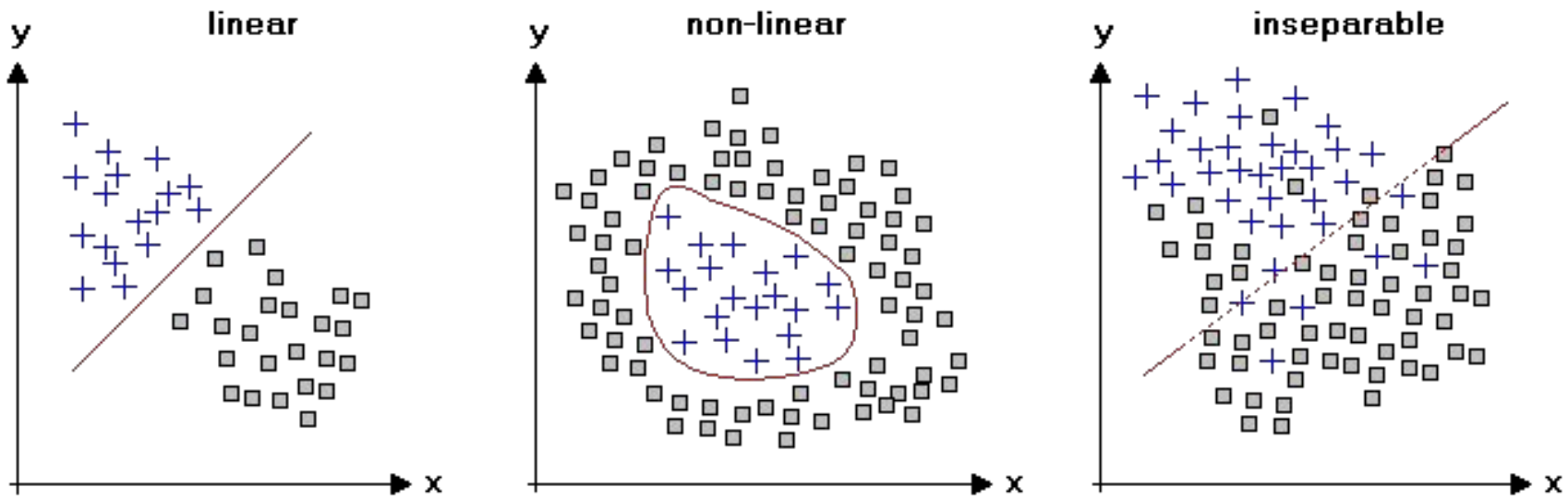
- "SVM Tutorial" Z. Gavrilov
- "Support Vector Machines: A Simple Tutorial" A. Nefedov
- "Introduction to Support Vector Machines" D. Boswell
- "A Gentle Introduction to Support Vector Machines in Biomedicine" A. Statnikov et al.
- "Genomic Similarity and Kernel Methods I" D. Schaid
- "Genomic Similarity and Kernel Methods II" D. Schaid

Klasyfikatory liniowe

Klasyfikatory liniowe (linear classifiers)

- **granica decyzyjna (decision boundary)** to hiperpłaszczyzna
- zakłada, że klasy są **liniowo separowalne (linearly separable)**
  - gdy nie są, to popełnia błąd - wybieramy rozwiązanie o najmniejszym koszcie
  - gdy są, to istnieje **nieskończenie wiele** rozwiązań o zerowym koszcie
- **przykłady:**
  - regresja logistyczna, liniowy SVM
  - binomial / multinomial Naive Bayes, Linear Discriminant Analysis
- **typowo:** high bias, low variance

Liniowa separowalność



Klasyfikatory liniowe - zalety i wady

Zalety:

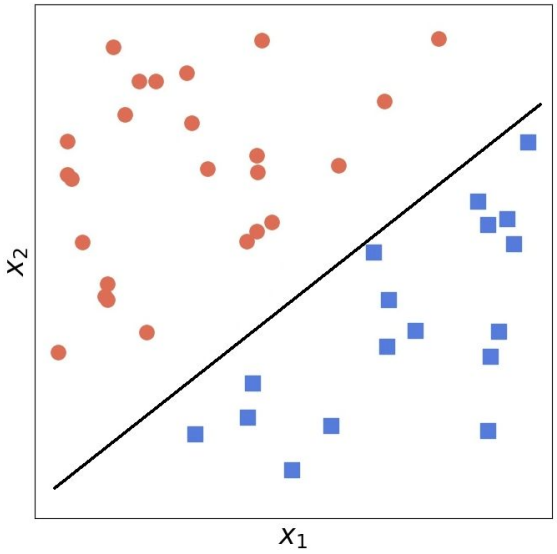
- proste
- bardzo szybkie
- bardzo skalowalne
- dobrze działają w przestrzeniach wysokowymiarowych
- łatwo regularyzować

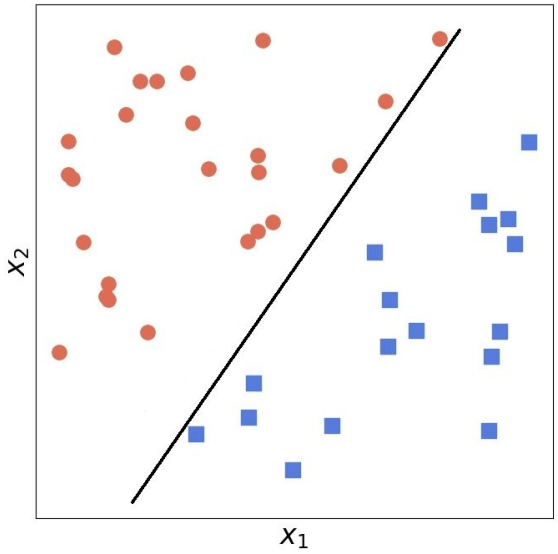
Wady:

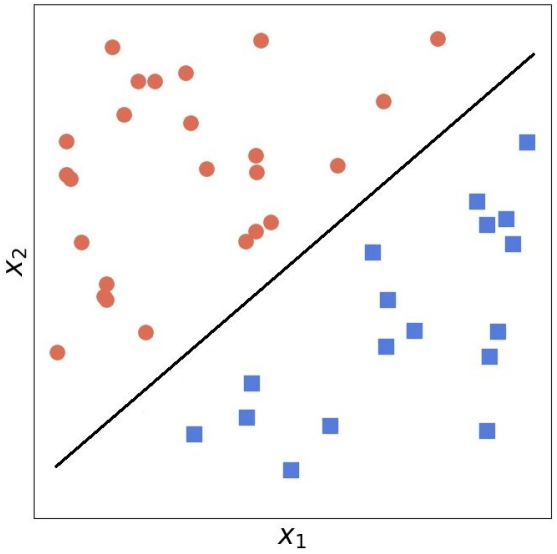
- często przeciętne wyniki
- wymagają dobrej inżynierii cech

Klasy liniowo separowalne

Pytanie: czy są lepsze i gorsze rozwiązania, jeżeli klasy są liniowo separowalne?



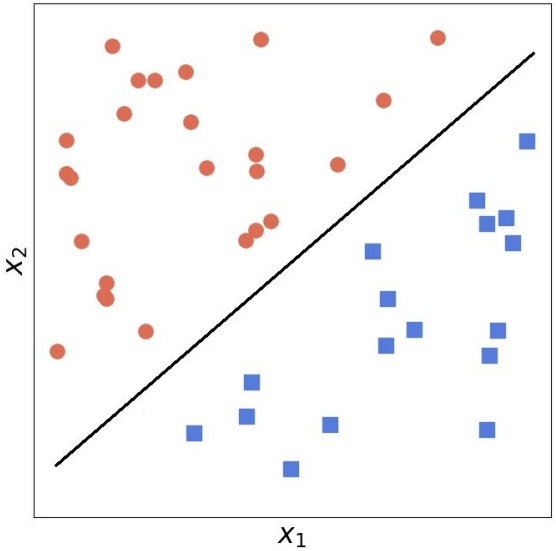




Liniowy SVM

Maximal margin classifier (MMC)

- **idea:** "najbezpieczniejszy" klasyfikator to taki, który daje sobie największy margines bezpieczeństwa
- powinien być najbardziej odporny na outliery, szum itp.
- **margines (margin)** to odległość do najbliższych punktów z przeciwnych klas
- **Maximal Margin Classifier (MMC):**
  - klasyfikator liniowy
  - granica decyzyjna o największym marginesie
  - zakłada liniową separowalność klas



Oznaczenia

- **model liniowy** do klasyfikacji o postaci:

$$f(x) = x^T w + b$$

w - parametry

b - intercept (bias)

- klasyfikacja dla klas {-1, +1}:

$$\hat{y}_i = \mathrm{sign}(x_i^T w + b)$$

- **uwaga:** w dalszej części oznaczamy *i*-tą próbkę indeksem dolnym *i*

i = 1, …, N

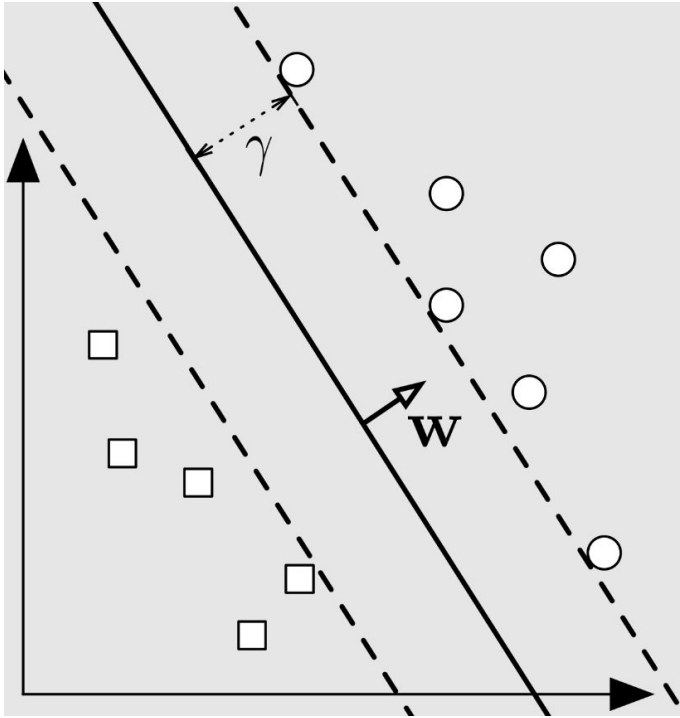
N - rozmiar datasetu

Margines (margin)

- **margines (margin)** - odległość od granicy decyzyjnej do najbliższych punktów z obu stron
- leży na środku między klasami - "najbezpieczniejsze" z obu stron
- granica decyzyjna to hiperpłaszczyzna o wzorze:

$$x^T w + b = 0$$

- odległość punktu od granicy to **pewność (confidence)**
- punkty najbliżej marginesu są najbardziej niepewne



Functional margin

- **wynik (score)** klasyfikatora:

  $f(x_i) = x_i^T w + b$ -> zakres wartości (-∞, +∞)

- **predykcja** klasyfikatora:

  $\hat{y}_i = \mathrm{sign}(x_i^T w + b)$ -> zakres wartości {-1, +1}

- **functional margin** to miara pewności klasyfikatora co do predykcji:

  $$\tilde{\gamma}_i = y_i * f(x_i) = y_i * (x_i^T w + b)$$

Functional margin

$$\tilde{\gamma}_i = y_i * f(x_i) = y_i * (x_i^T w + b)$$

- zakres wartości (-∞, +∞)
- **wysoki, gdy:**
  - prawidłowa klasyfikacja - dobry znak
  - wysoki wynik - duża pewność
- **problem:** co, jeżeli zrobimy α razy większe wagi?

$$\alpha\tilde{\gamma}_i = y_i * (\alpha x_i^T w + \alpha b) = y_i * \alpha * (x_i^T w + b)$$

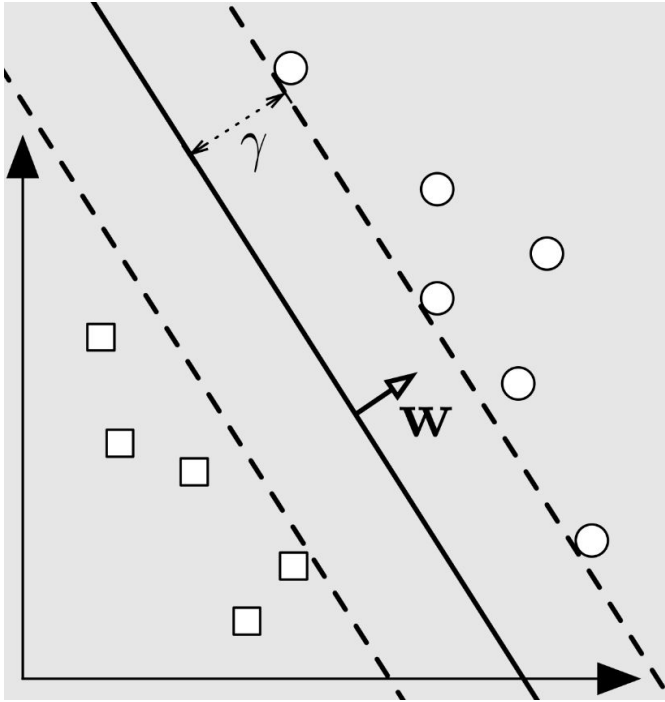
- hiperpłaszczyzna ta sama, a "pewność" większa!

Geometric margin

- **idea:** przeskalujmy functional margin
- **geometric margin**

$$\gamma_i = y_i \left(\frac{x_i^T w + b}{||w||} \right)$$

- odpowiada **odległości punktu** x_i od granicy decyzyjnej
- dowód



Maximal margin classifier (MMC)

- **obserwacja:** margines dla całego zbioru to najmniejsza odległość od granicy decyzyjnej do punktu
- **wniosek:** trzeba zmaksymalizować najmniejszy margines
- **trening** MMC to rozwiązanie problemu **optymalizacji matematycznej:**

$$\underset{w,\, b,\, \gamma}{\text{maximize}} \quad \gamma$$

$$\text{subject to} \quad y_i \frac{x_i^T w + b}{||w||} \geq \gamma \text{ for all } i$$

MMC - optymalizacja

$$\underset{w,b,\gamma}{\text{maximize}} \quad \gamma$$

$$\text{subject to} \quad y_i \frac{x_i^T w + b}{||w||} \geq \gamma \text{ for all } i$$

- **cel optymalizacji:** maksymalny margines γ
- **zmienne:** wagi hiperpłaszczyzny w, b; margines γ
- **ograniczenia:** każdy punkt ma leżeć po dobrej stronie oraz być w odległości co najmniej marginesu γ od granicy decyzyjnej

Uproszczenie optymalizacji

- **uprośćmy to:** będzie szybciej

$$\underset{w,b,\gamma}{\text{maximize}} \quad \gamma$$

$$\text{subject to} \quad y_i \frac{x_i^T w + b}{||w||} \geq \gamma \text{ for all } i$$

- typowo eliminujemy zmienne, dzielenie, warunki etc.
- częsty krok w optymalizacji matematycznej

MMC - uproszczenie optymalizacji

$$\underset{w,b,\gamma}{\text{maximize}} \quad \gamma$$

$$\text{subject to} \quad y_i \frac{x_i^T w + b}{||w||} \geq \gamma \text{ for all } i$$

- mnożymy ograniczenie stronami przez ||w||

$$\underset{w,b,\gamma}{\text{maximize}} \quad \gamma$$

$$\text{subject to} \quad y_i(x_i^T w + b) \geq \gamma ||w|| \text{ for all } i$$

- **problem:** można pomnożyć przez dowolne α > 0 i dalej będzie prawidłowo
- nieskończenie wiele rozwiązań - nadmiarowe **stopnie swobody (degrees of freedom)**

MMC - uproszczenie optymalizacji

- **idea:** nałóżmy ograniczenie, które wybierze nam jedno, unikalne rozwiązanie

$$||w|| = \frac{1}{\gamma} \longrightarrow \gamma = \frac{1}{||w||}$$

- wolno nam, bo taki wektor też spełnia rozwiązanie, dla jakiegoś α
- otrzymujemy wtedy:

$$\begin{aligned} \underset{w,\,b}{\text{maximize}} \quad & \frac{1}{||w||} \\ \text{subject to} \quad & y_i(x_i^T w + b) \geq 1 \text{ for all } i \end{aligned}$$

- **obserwacja:** pozbyliśmy się γ, więc mamy mniej zmiennych i prostszy problem!

Konsekwencje geometryczne

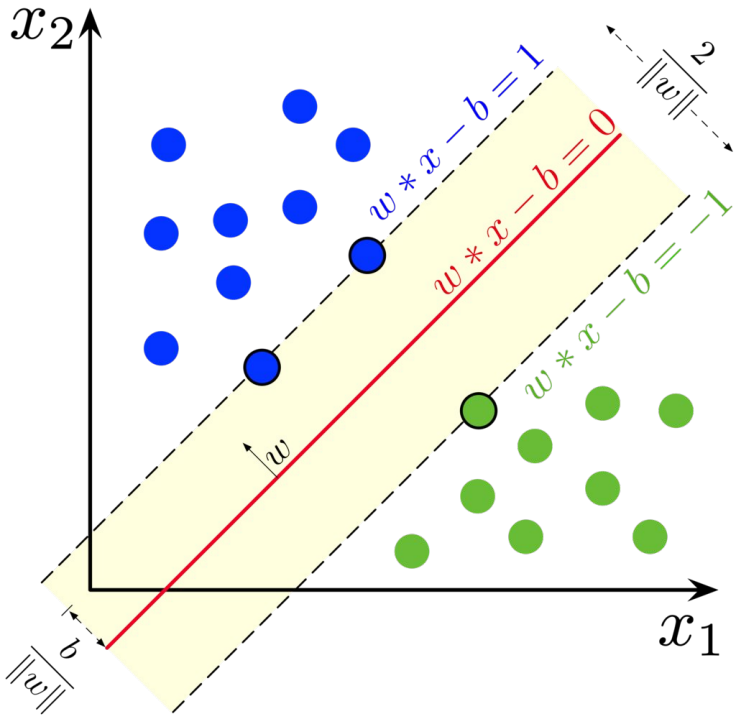
- mamy teraz:

$$\underset{w,b}{\text{maximize}} \quad \frac{1}{||w||}$$

$$\text{subject to} \quad y_i(x_i^T w + b) \geq 1 \text{ for all } i$$

- **obserwacja:** wprost maksymalizujemy margines

$$\gamma = \frac{1}{||w||}$$



MMC - uproszczenie optymalizacji

$$\underset{w,b}{\text{maximize}} \quad \frac{1}{||w||}$$

$$\text{subject to} \quad y_i(x_i^T w + b) \geq 1 \text{ for all } i$$

- **twierdzenie:** maximum 1/x to minimum x
- **twierdzenie:** można używać dowolnych funkcji monotonicznych do przekształceń dla nieujemnych zmiennych

$$\underset{w,b}{\text{minimize}} \quad \frac{1}{2}||w||^2$$

$$\text{subject to} \quad y_i(x_i^T w + b) \geq 1 \text{ for all } i$$

- **prosty problem:** wyrażenie do optymalizacji jest różniczkowalne i wypukłe

MMC - programowanie kwadratowe

- ostateczna postać optymalizacji dla maximum margin classifier to:

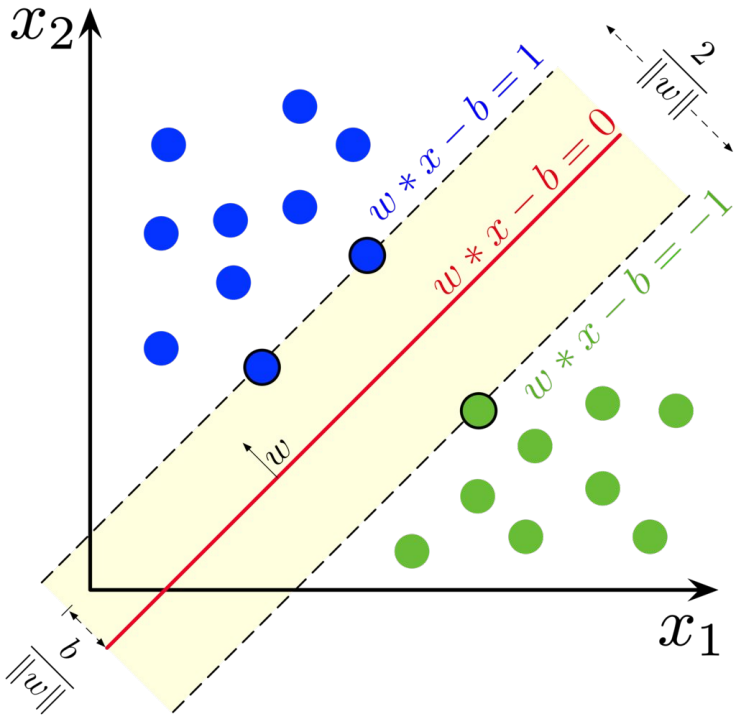
$$\underset{w,b}{\text{minimize}} \quad \frac{1}{2}||w||^2$$

$$\text{subject to} \quad y_i(x_i^T w + b) \geq 1 \text{ for all } i$$

- jest to problem **programowania kwadratowego (quadratic programming):**
  - cel: funkcja kwadratowa
  - ograniczenia: liniowe
- **bardzo dobrze zbadany problem**, istnieją bardzo szybkie solwery

Support vectors

- dla całego zbioru margines leży na środku pomiędzy klasami
- **obserwacja:** znaczenie mają tylko punkty leżące na marginesie, na jego brzegach
- punkt = wektor
- te punkty to **wektory nośne (support vectors)** - bo "utrzymują" margines



Problem z maximal margin classifier

- **problem:** MMC zakłada, że klasy są liniowo separowalne
- gdy nie są, to wyrzuci błąd - nie spełni ograniczeń
- MMC nazywa się też **hard margin classifier** - trzeba bezwzględnie spełnić wszystkie warunki

$$\underset{w,b}{\text{minimize}} \quad \frac{1}{2}||w||^2$$

$$\text{subject to} \quad y_i(x_i^T w + b) \geq 1 \text{ for all } i$$

- **idea:** pozwolić na łamanie części ograniczeń

Liniowy SVM

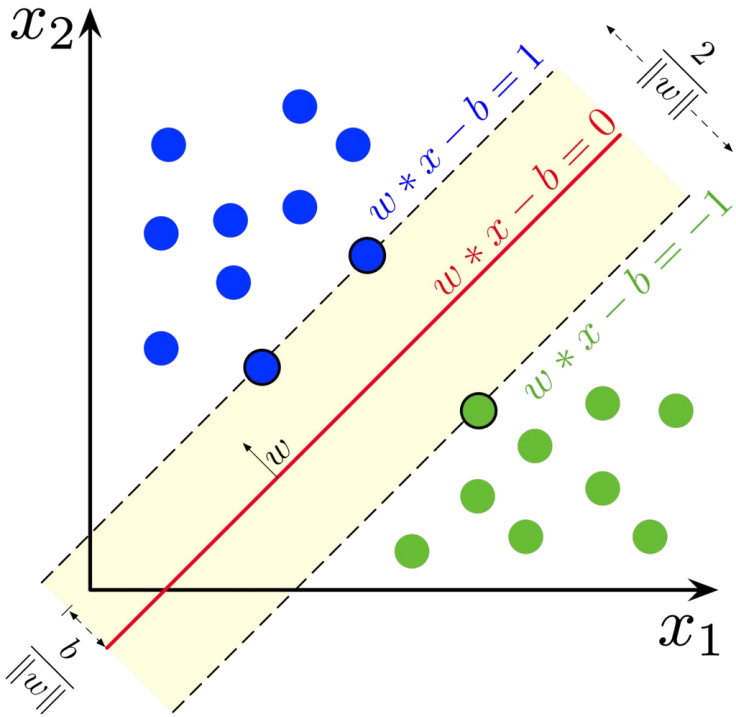
- **liniowy Support Vector Machine (SVM)**
- **idea:** pozwolić na łamanie części ograniczeń, "złagodzony" MMC
- nazywane też **soft margin classifier**
- pozwalamy, żeby punkty były:
  - wewnątrz marginesu, albo nawet po złej stronie
  - ale żeby było takich jak najmniej
- **funkcja kosztu:** hinge loss

Liniowy SVM

- **jak to zapisać** w programowaniu kwadratowym?
- zmienne pomocnicze, tzw. **slack variables**

$$y_i(x_i^T w + b) \geq 1 - \xi_i$$

  - idealna klasyfikacja (margines >= 1): ξ_i = 0
  - naruszenie marginesu: 0 < ξ_i <= 1
  - błędna klasyfikacja: ξ_i > 1
- **minimalizujemy ξ_i**
- koszt za wchodzenie w margines



Liniowy SVM - problem optymalizacyjny

- postać problemu optymalizacyjnego:

$$\begin{aligned} \underset{w,b,\xi}{\text{minimize}} \quad & \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(x_i^T w + b) \geq 1 - \xi_i \text{ for all } i, \\ & \xi_i \geq 0 \end{aligned}$$

- C - stała, która reguluje, na czym nam bardziej zależy - małym ||w||^2, czy małych ξ_i
- **uprośćmy to:** ponownie, będzie szybciej

Liniowy SVM - uproszczenie problemu

$$\begin{aligned} \underset{w,b,\xi}{\text{minimize}} \quad & \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(x_i^T w + b) \geq 1 - \xi_i \text{ for all } i, \\ & \xi_i \geq 0 \end{aligned}$$

- zamieniamy wyrażenia w nierówności miejscami:

$$\begin{aligned} \underset{w,b,\xi}{\text{minimize}} \quad & \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & \xi_i \geq 1 - y_i(x_i^T w + b) \text{ for all } i, \\ & \xi_i \geq 0 \end{aligned}$$

Liniowy SVM - uproszczenie problemu

$$\begin{aligned} \underset{w,b,\xi}{\text{minimize}} \quad & \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & \xi_i \geq 1 - y_i(x_i^T w + b) \text{ for all } i, \\ & \xi_i \geq 0 \end{aligned}$$

Możliwe wartości ξ_i:

- idealna klasyfikacja -> $\xi_i = 0$
- w przeciwnym razie -> $\xi_i = 1 - y_i(x_i^T w + b)$

Wniosek: $\xi_i = \max(1 - y_i(x_i^T w + b), 0)$

Liniowy SVM - uproszczenie problemu

- mamy: $\xi_i = \max(1 - y_i(x_i^T w + b), 0)$
- podstawiamy:

$$\begin{aligned} \underset{w,b,\xi}{\text{minimize}} \quad & \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & \xi_i \geq 1 - y_i(x_i^T w + b) \text{ for all } i, \\ & \xi_i \geq 0 \end{aligned}$$

$$\downarrow$$

$$\underset{w,b}{\text{minimize}} \quad \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \max(1 - y_i(x_i^T w + b), 0)$$

Liniowy SVM - uproszczenie problemu

- dostaliśmy problem **optymalizacji bez ograniczeń (unconstrained optimization)**

$$\underset{w,b}{\text{minimize}} \quad \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\max(1 - y_i(x_i^T w + b), 0)$$

- takie problemy są **o wiele** prostsze!
- dzielimy przez C - wolno nam, bo to stała i C > 0

$$\underset{w,b}{\text{minimize}} \quad \sum_{i=1}^{n}\max(1 - y_i(x_i^T w + b), 0) + \frac{1}{2C}||w||^2$$

- czy coś nam to przypomina?

Liniowy SVM - primal problem

- mamy zwykły problem **minimalizacji funkcji kosztu z regularyzacją!**

$$\underset{w,b}{\text{minimize}} \quad \underbrace{\sum_{i=1}^{n} \max(1 - y_i(x_i^T w + b), 0)}_{\text{hinge loss}} + \underbrace{\frac{1}{2C}||w||^2}_{\text{L2 regularization}}$$

- **hinge loss** to funkcja kosztu dla klasyfikacji, która penalizuje tylko naruszenia marginesu
- jeżeli punkt jest "bezpieczny" (na marginesie albo poza nim), to ma koszt 0, niezależnie od tego, jak daleko jest
- takie sformułowanie nazywa się **primal problem**

Liniowy SVM - regularyzacja

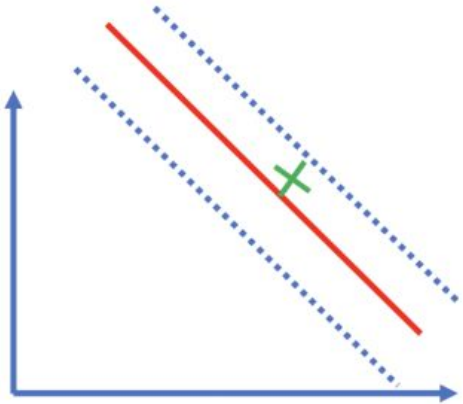
$$\underset{w,b}{\text{minimize}} \quad \underbrace{\sum_{i=1}^{n} \max(1 - y_i(x_i^T w + b), 0)}_{\text{hinge loss}} + \underbrace{\frac{1}{2C}||w||^2}_{\text{L2 regularization}}$$

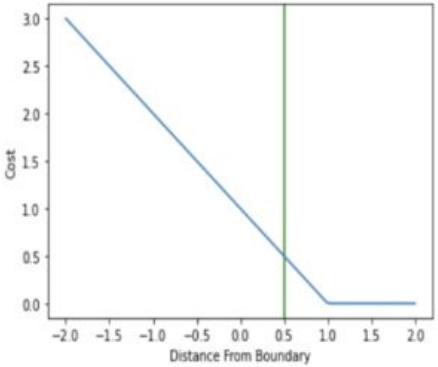
- C oznacza **odwrotność siły regularyzacji**
- można też zapisać "normalnie" siłę regularyzacji λ: $\lambda = \frac{1}{C}$
- Scikit-learn używa notacji z C
- **obserwacja:** słabsza regularyzacja to więcej support vectors
- **szybsze** dla silniejszej regularyzacji

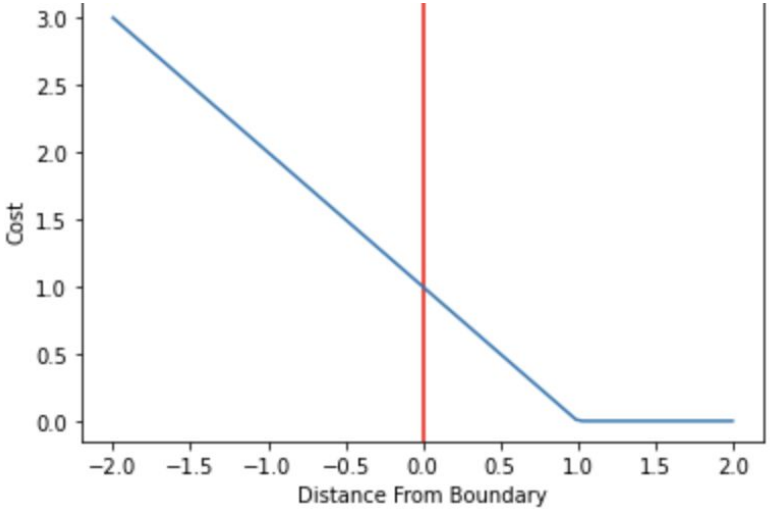
Hinge loss

- odpowiada maksymalizacji marginesu
- **zalety:** wypukła, odporna na outliery
- **wady:** nieróżniczkowalna

$$L_{hinge}(w) = \max(1 - y * w^T x, 0)$$







Liniowy SVM - primal optimization

$$\underset{w,b}{\text{minimize}} \quad \underbrace{\sum_{i=1}^{n} \max(1 - y_i(x_i^T w + b), 0)}_{\text{hinge loss}} + \underbrace{\frac{1}{2C}||w||^2}_{\text{L2 regularization}}$$

- **primal problem / primal optimization**
- używa się wariantu coordinate descent (CD) - [artykuł dla zainteresowanych]
- **wymaga normalizacji cech** - bo mamy regularyzację

- predykcja: $\hat{y}_i = \text{sign}(w^T x_i + b)$

Liniowy SVM - złożoność obliczeniowa

- **złożoność treningu:** zależy, ile zajmie solwerowi
- **złożoność predykcji:** O(d)

- szybkość treningu **bardzo** zależy od siły regularyzacji
- predykcja jest **niesamowicie** szybka

Liniowy SVM - zalety i wady

Zalety:

- prostota, szybkość, skalowalność
- robust - odporny na szum, outliery itp.
- interpretowalny - jak modele liniowe
- bardzo dobre wyniki jak na klasyfikator liniowy

Wady:

- tylko klasyfikacja binarna
- brak prawdopodobieństw
- czuły na odwrotność siły regularyzacji C
- tylko regularyzacja L2

SVM dual

Podejścia primal i dual

- w optymalizacji matematycznej wyjściowy problem to **primal**, dla wektora zmiennych w
- każdy problem ma także alternatywne sformułowanie, tzw. **dual**
- w przypadku wielu problemów, w tym SVM, daje dokładnie to samo rozwiązanie
- do przekształcenia służy **metoda mnożników Lagrange'a**

- **po co?**
- problem dualny może być prostszy, szybszy, lepiej się skalować, łatwiej wprowadzić do niego dodatkowe warunki etc.

Metoda Lagrange'a

Metoda (mnożników) Lagrange'a:

- metoda optymalizacji matematycznej
- znajduje ekstrema warunkowe
- daje rozwiązanie dokładne, jeżeli funkcja jest wypukła (convex)

Przykład:

Znajdź minimum funkcji:

`f(x,y) = x+y`

Przy warunku:

`x^2 + y^2 = 1`

Metoda Lagrange'a - działanie

- dodajemy **mnożniki Lagrange'a** α
- dodatkowe zmienne, tyle, ile punktów w zbiorze
- wyprowadzenie jest relatywnie proste, ale długie
- wykład 10 - wyprowadzenie (dla zainteresowanych)

  (albo kurs na Udemy z początku prezentacji)

SVM primal

$$\underset{w,b}{\text{minimize}} \quad \sum_{i=1}^{n} \max(1 - y_i(x_i^T w + b), 0) + \frac{1}{2C}||w||^2$$

$$\downarrow$$

SVM dual

$$\underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j * y_i y_j * (x_i \cdot x_j)$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \text{ for all } i,$$

$$0 \leq \alpha_i \leq C$$

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i$$

SVM dual - po co?

$$\underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j * y_i y_j * (x_i \cdot x_j)$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \text{ for all } i,$$

$$0 \leq \alpha_i \leq C$$

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i$$

- **w sumie… po nic** - zasadniczo przez 31 lat istnienia liniowego SVM nikt tego nie ruszał
- **zalety:**
  - świetnie skaluje się z wymiarem
  - α jest niezerowe tylko dla support vectors
- **wada:** złożoność O(n^2), wolne programowanie kwadratowe (dużo ograniczeń)

SVM dual - obserwacje

$$\underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \underline{\alpha_i \alpha_j} * y_i y_j * \underline{(x_i \cdot x_j)}$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \text{ for all } i,$$

$$0 \leq \alpha_i \leq C$$

- **obserwacja 1:** używamy tylko iloczynów skalarnych wektorów x
- **obserwacja 2:** wagi α występują parami

Konsekwencje tych obserwacji

- używamy tylko iloczynów skalarnych wektorów x:
  - C. Cortes, V. Vapnik *"Support-vector networks"*
  - **kernel SVM -** klasyfikator nieliniowy

- wagi α występują parami:
  - J. Platt *"Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines"*
  - **szybki trening kernel SVM** - bez programowania kwadratowego

Kernel SVM

Twierdzenie Covera

- **twierdzenie Covera (Cover theorem):**

"Problem klasyfikacji po nieliniowym przekształceniu do wysokowymiarowej przestrzeni jest z dużym prawdopodobieństwem liniowo separowalny."

- **błogosławieństwo wymiarowości (blessing of dimensionality)**
- działa, bo nie obchodzi nas sąsiedztwo ani odległości, tylko liniowa separowalność!

- dlatego działa dodawanie np. wielomianów cech

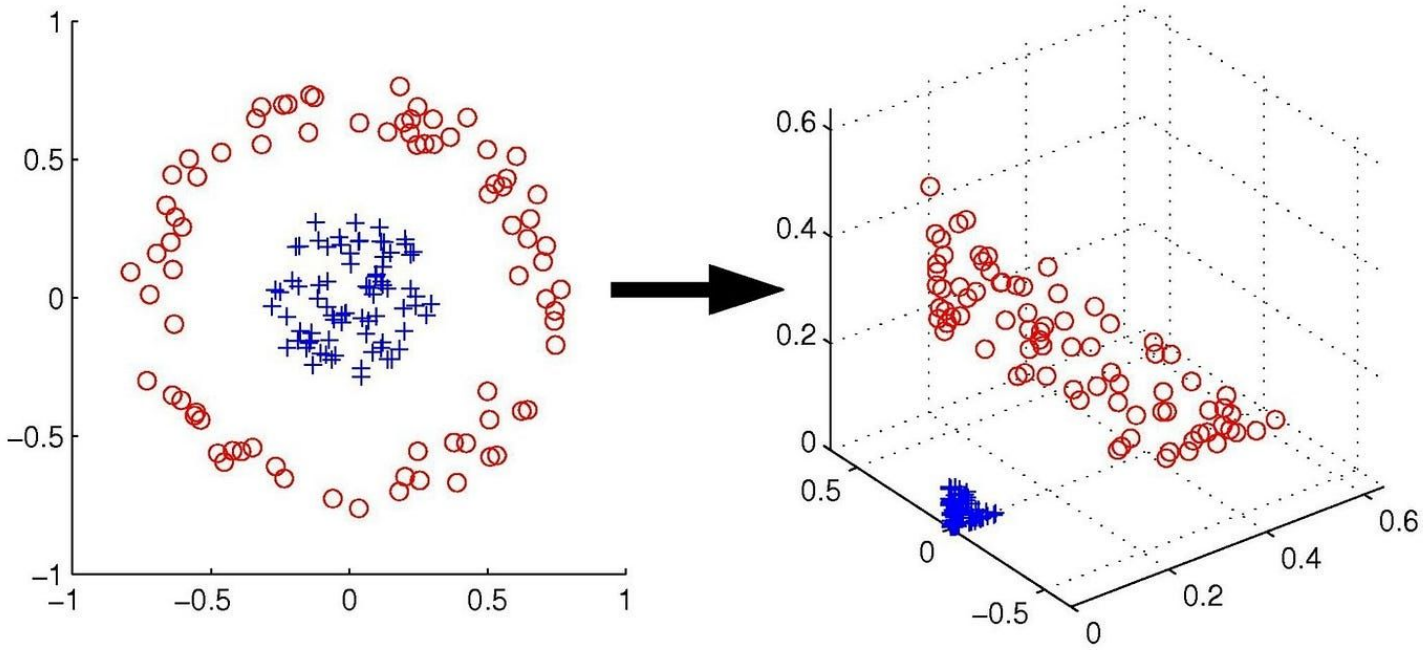
Nieliniowe przekształcenia cech

- żeby zmienić klasyfikator liniowy w nieliniowy (w pierwotnej przestrzeni), można dodać **nieliniowe przekształcenia cech**

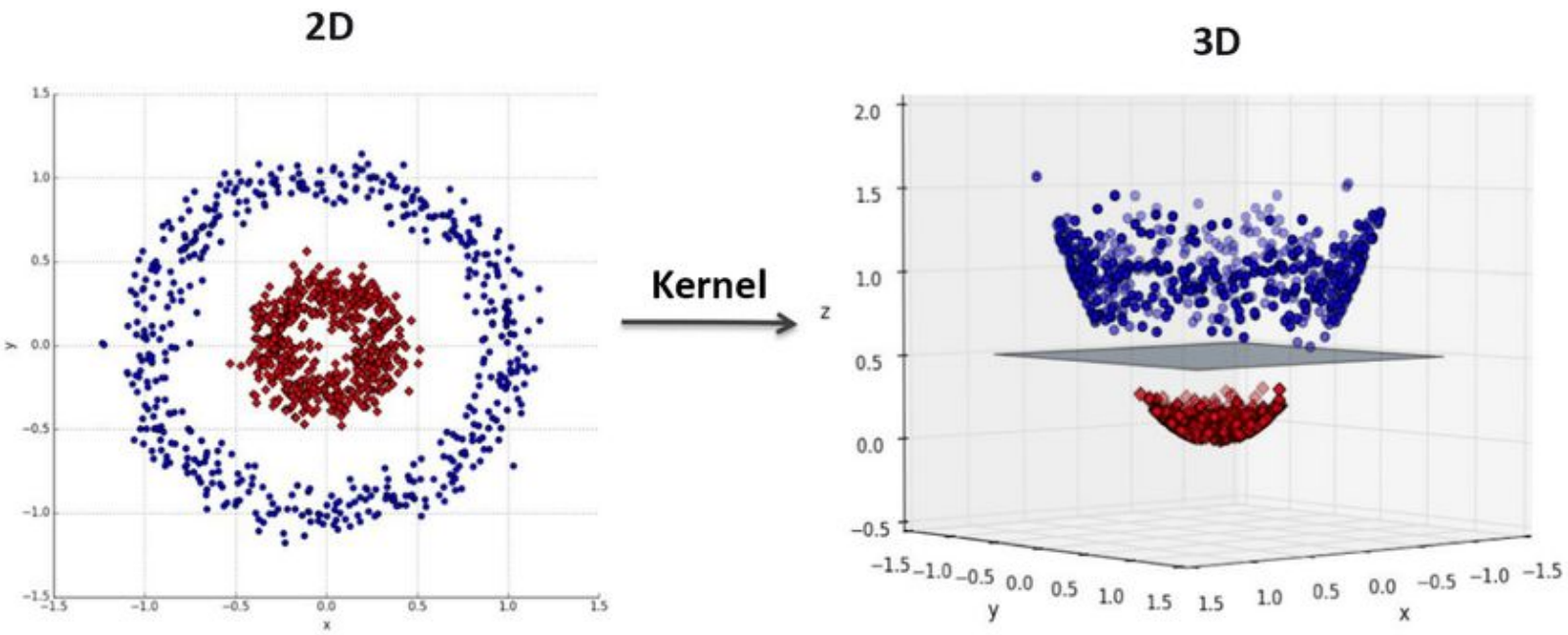
$$x_i \to \varphi(x_i)$$

- np. wielomiany: [a, b] -> [a, b, a^2, b^2, ab]
- **złożoność eksploduje:** 100 cech, wielomian 2 stopnia -> ponad 5000 cech!
- **chcemy:**
  - bardzo wysokowymiarowych przestrzeni i dobrych cech
  - ale nie chcemy tego liczyć ani przechowywać

Blessing of dimensionality



Blessing of dimensionality



Nieliniowy SVM

- **obserwacja:** w dual SVM obchodzi nas tylko iloczyn skalarny z nowymi cechami

$$\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j * y_i y_j * (x_i^T x_j) \longrightarrow \sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j * y_i y_j * (\varphi(x_i)^T \varphi(x_j))$$

- wystarcza wartość iloczynu skalarnego w przestrzeni wysokowymiarowej
- iloczyn skalarny = funkcja **podobieństwa**
- da się użyć **funkcji jądrowych (kernel functions)** - szczególnej rodziny podobieństw

Kernel function

- **funkcja jądrowa (kernel function)** pozwala obliczyć iloczyn skalarny w innej przestrzeni bez jawnego obliczania cech

$$K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$$

- **nie musimy:** obliczać nowych cech, zużywać pamięci, robić iloczynu skalarnego długich wektorów
- dostajemy po prostu wartość jednym prostym wzorem - tzw. **kernel trick**
- kernel to **miara podobieństwa** wektorów, jak iloczyn skalarny, ale od razu w przestrzeni po przekształceniu

Kernel matrix

- **macierz kernelowa (kernel matrix, Gram matrix)**
- macierz podobieństw każdy-z-każdym (*n* x *n*), jawnie reprezentuje zależności między punktami w danych
- **wszystkie** metody kernelowe działają na tej macierzy, **nie** wprost na danych ze zbioru treningowego!

$$K(X) = \begin{bmatrix} \varphi(x_1, x_1) & \varphi(x_1, x_2) & \dots & \varphi(x_1, x_n) \\ \varphi(x_2, x_1) & \varphi(x_2, x_2) & \dots & \varphi(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi(x_n, x_1) & \varphi(x_n, x_2) & \dots & \varphi(x_n, x_n) \end{bmatrix}$$

Kernele

- **liniowy (linear):**

$$K_{linear}(x_i, x_j) = x_i^T x_j$$

  - zwykły iloczyn skalarny, używaliśmy kerneli cały czas!

- **wielomianowy (polynomial):**

$$K_{poly}(x_i, x_j) = (x_i^T x_j + c)^p$$

  - wielomian p-tego stopnia
  - c - stała, rzadko używany parametr

Kernele

- **Radial Basis Function (RBF):**

$$K_{RBF}(x_i, x_j) = \exp\left(-\frac{||x_i - x_j||^2}{\gamma}\right)$$

  - najczęściej używany
  - typowo daje najlepsze wyniki
  - jeżeli γ jest równa wariancji cech (σ^2), to nazywa się **Gaussian kernel** (domyślne w Scikit-learn)
  - rzutuje punkty do **przestrzeni nieskończeniewymiarowej** - dowód

Inne kernele

- metody kernelowe **naturalnie** działają dla **nie-tabelarycznych** danych
- graph kernels
- neural tangent kernel (NTK)
- min-max kernel
- histogram intersection kernel
- DNA kernels
- …

Kernel SVM

- **nieliniowy** w oryginalnej przestrzeni
- **liniowy** w wysokowymiarowej przestrzeni, którą tworzy kernel
- kernel SVM = liniowy SVM + sprytna sztuczka, żeby liczyć go "gdzie indziej"!

- opiera się **wyłącznie** o podobieństwa punktów w wysokowymiarowej przestrzeni

Kernel SVM - trening

- **kernel dual SVM:**

$$\underset{\alpha}{\text{maximize}} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j * y_i y_j * K(x_i, x_j)$$

$$\text{subject to} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \text{ for all } i,$$

$$0 \leq \alpha_i \leq C$$

- **żadnych zmian** w algorytmie!
- dual jest kluczowy - same iloczyny wektorów, złożoność nie zależy od wymiaru z kernela

Kernel SVM - predykcja

$$\hat{y}_i = \text{sign}\left(b + \sum_{j=1}^{n_{sv}} \alpha_j y_j K(x_i, x_j)\right)$$

- **obliczamy tylko dla support vectors** - tylko tam α są niezerowe
- obliczamy kernel między nowym x_i, a wektorami nośnymi x_j
- wolniejsze od liniowego SVM, ale dalej dość szybkie

Kernel SVM i normalizacja cech

$$K_{RBF}(x_i, x_j) = \exp\left(-\frac{||x_i - x_j||^2}{\gamma}\right)$$

- **kernel SVM wymaga normalizacji cech!**
- bo mamy odległości między próbkami
- standaryzacja vs min-max scaling - ciężko stwierdzić

Metody kernelowe

- trzeba obliczyć **macierz jądrową (kernel matrix):** $X_{ij} = K(x_i, x_j)$
- **inputem** jest tak naprawdę macierz wartości kerneli, a nie cech (ale np. Scikit-learn to policzy pod spodem)
- **minimalna złożoność:** O(n^2)
- trzeba obliczyć macierz każdy-do-każdego (n*n)
- **typowe cechy:**
  - potężna siła wyrazu i nieliniowości
  - duży koszt obliczeniowy

Metody kernelowe

- kernel ridge regression
- kernel SVM
- kernel logistic regression
- kernel Random Forest
- kernel PCA
- …

Kernel SVM - złożoność obliczeniowa

- trening - obliczenie kernel matrix + optymalizacja
- **złożoność treningu:** między O(n^2) a O(n^3)
- w praktyce niestety często bliżej tego drugiego

- predykcja - obliczenie kerneli z support vectorami + suma ważona
- **złożoność predykcji:** O(n_sv)
- ale niestety duża stała

Kernel SVM - złożoność pamięciowa

- **złożoność pamięciowa:** O(n^2)

- **w praktyce:** typowo używa się cache'a wybranego rozmiaru
- tylko czasem używamy elementu kernel matrix więcej niż raz
- albo można zapisać macierz na dysk i czytać kawałki

SVM a parametryczność

- **liniowy SVM jest parametryczny** - zwykły model liniowy (ale nie zakłada żadnych rozkładów!)

- **kernel SVM jest nieparametryczny**
- wszystko zależy od liczby support vectors, która jest:
  - nieznana z góry
  - nie da się nią sterować bezpośrednio
  - zależy od danych

Kernel SVM - zalety i wady

Zalety:

- bardzo dobre wyniki
- nie wymaga dużo danych
- dobrze działa przy danych wysokowymiarowych

Wady:

- tylko klasyfikacja binarna
- brak prawdopodobieństw
- bardzo czuły na siłę regularyzacji C
- duży koszt treningu
- sekwencyjny (algorytm SMO)
- brak możliwości regularyzacji L1

Podsumowanie

Support Vector Machine (SVM)

- **wersje:**
  - liniowy / kernel (nieliniowy)
  - primal / dual
- **hiperparametry:** odwrotność siły regularyzacji C, typ i hiperparametry kernela
- **wymaga normalizacji cech:** standaryzacja, min-max scaling
- **złożoność zależy od regularyzacji:** silniejsza regularyzacja (mniejsze C) jest szybsza

SVM - zalety

- **bardzo dobre wyniki** - liniowy niezłe, kernelowy świetne
- **szybki trening** - dla liniowego SVM
- **działa dobrze dla wielu wymiarów**
- **kernele dla różnych danych** - bardzo ogólna metoda

SVM - wady

- **nieinterpretowalny** - nie w wersji kernelowej
- **tylko klasyfikacja binarna** - są rozszerzenia, ale kosztowne
- **bez prawdopodobieństw** - są rozszerzenia, ale kosztowne
- **łatwo overfituje + duża czułość na dobór hiperparametrów** - trzeba tune'ować
- **duża złożoność obliczeniowa i pamięciowa** - jak metody kernelowe
- **sekwencyjny** - przynajmniej wszystkie typowe implementacje

Slajdy bonusowe

SVM do regresji

Uwaga o nazewnictwie

- SVM nadaje się i do klasyfikacji, i do regresji
- czasem nazywa się je konkretnie:
  - **Support Vector Classification (SVC)**
  - **Support Vector Regression (SVR)**
- domyślnie używa się wersji kernel dual
- jeżeli ktoś używa liniowego SVM (właściwie zawsze primal), to oznacza się Linear SVC albo Linear SVR
- tej notacji używa też Scikit-learn

Support Vector Regression (SVR)

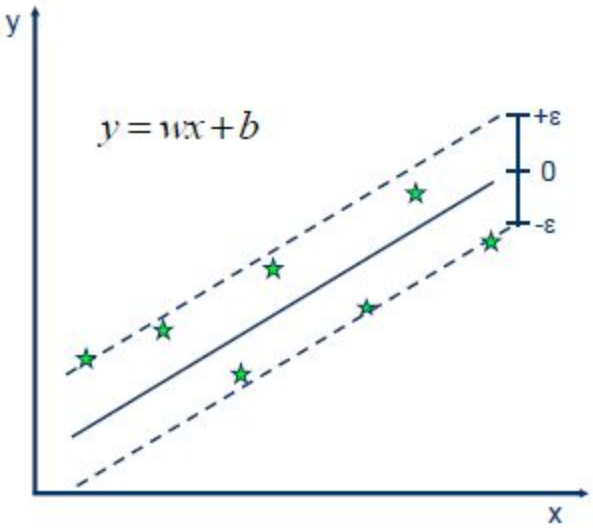
- w regresji mamy wartość ciągłą, a dalej chcemy używać SVM
- trzeba zdefiniować **margines dla regresji**
- **idea:** zróbmy margines "dookoła" prawdziwej wartości, i jeżeli wartość przewidywana przez model wpada w ten margines, to jest ok
- robi to **funkcja kosztu ε-insensitive:**

$$L_\epsilon = \begin{cases} 0 & \text{if } |y - \hat{y}| \leq \epsilon \\ |y - \hat{y}| - \epsilon & \text{otherwise} \end{cases}$$

$$L_\epsilon = \max(|y - \hat{y}| - \epsilon, 0)$$

Liniowy SVR

- **predykcja:** $\hat{y}_i = w^T x_i + b$
- większy ε = mocniejsza regularyzacja
- im większy margines, tym bardziej luzujemy nasze ograniczenia
- **ale:** to zakłada, że taki warunek da się spełnić dla wszystkich punktów
- ten sam problem, co z separowalnością liniową



- Solution:

$$\min \frac{1}{2}\|w\|^2$$

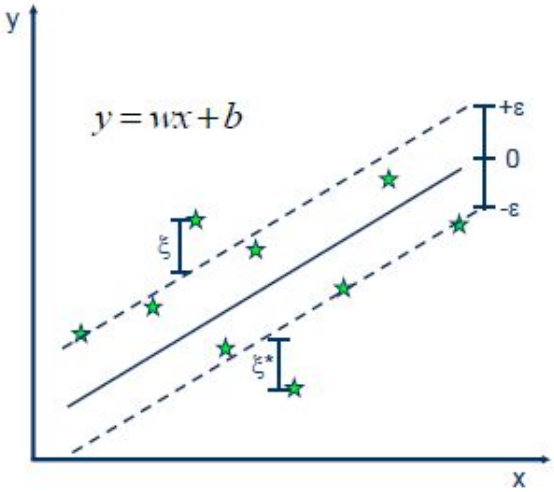
- Constraints:

$$y_i - wx_i - b \le \varepsilon$$

$$wx_i + b - y_i \le \varepsilon$$

Liniowy SVR

- **idea:** dodajemy znowu slack variables ξ
- teraz potrzebujemy ich 2 razy więcej, bo regresja może mieć za duże i za małe wartości
- w funkcji kosztu (primal SVR) nic się nie zmienia
- **liniowy SVR (Support Vector Regression):**



- Minimize:

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}(\xi_i + \xi_i^*)$$

- Constraints:

$$y_i - wx_i - b \le \varepsilon + \xi_i$$

$$wx_i + b - y_i \le \varepsilon + \xi_i^*$$

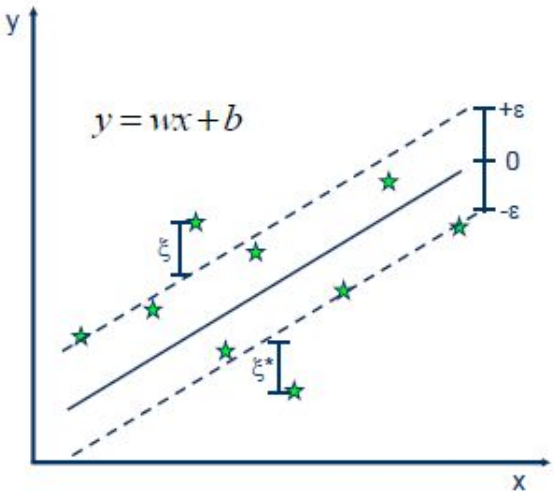
$$\xi_i, \xi_i^* \ge 0$$

$$\underset{w,b}{\text{minimize}} \quad \sum_{i=1}^{n} \max(|y_i - (w^T x_i + b)| - \epsilon, 0) + \frac{1}{2C}||w||^2$$

Liniowy SVR jako model liniowy

$$\underset{w,b}{\text{minimize}} \quad \sum_{i=1}^{n} \max(|y_i - (w^T x_i + b)| - \epsilon, 0) + \frac{1}{2C}||w||^2$$

- **funkcja kosztu:** ε-insensitive
- **algorytm:** wariant metody Newtona, tzw. line-search Newton method
- **zalety:** odporny na outliery
- **wady:** czuły na wartość ε, koszt obliczeniowy



- Minimize:

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}(\xi_i + \xi_i^*)$$

- Constraints:

$$y_i - wx_i - b \le \varepsilon + \xi_i$$
$$wx_i + b - y_i \le \varepsilon + \xi_i^*$$
$$\xi_i, \xi_i^* \ge 0$$

Kernel SVR

- wyprowadzenie wersji problemu dual dla SVR jest analogiczne do SVC
- na koniec zostaje problem programowania kwadratowego, też jak w SVC
- znowu są iloczyny skalarne wektorów x, znowu można użyć kerneli
- jedyna różnica - dwa razy więcej mnożnikóœ Lagrange'a, α i α* (ograniczają regresję z dołu i góry, dla zbyt małych / zbyt dużych predykcji)
- **finalna postać predykcji kernel SVR:**

$$\hat{y}_i = b + \sum_{j=1}^{n} (\alpha_j - \alpha_j^*) K(x_i, x_j)$$

SVM i klasyfikacja wieloklasowa

SVM a klasyfikacja wieloklasowa

- SVM potrafi działać tylko dla 2 klas
- co, gdy mamy K klas?
- **idea:** zróbmy wiele klasyfikatorów, gdzie każdy odróżnia "swoją" klasę od pozostałych

One-vs-Rest (OvR)

- **one-vs-rest (OvR)** - podejście, w którym odróżniamy daną klasę od pozostałych
- **trening:**
  - robimy K klasyfikatorów, po jednym dla klasy
  - przykłady z "jego" klasy są pozytywne
  - przykłady z pozostałych klas łączymy w klasę negatywną
- **predykcja:** zwracamy klasę o najwyższym wyniku
- "wieloklasowy SVM" to taki zbiór (ensemble) K modeli
- działa dla dowolnych modeli klasyfikacji binarnej: SVM, regresja logistyczna itd.

One-vs-One (OvO)

- **one-vs-rest (OvO)** - podejście, w którym odróżniamy pary klas od siebie
- **trening:**
  - robimy klasyfikator dla każdej pary klas, co daje K * (K - 1) / 2 klasyfikatorów
  - arbitralnie jedna klasa jest pozytywna, druga negatywna
- **predykcja:** każdy klasyfikator głosuje na jedną ze swoich dwóch klas, zwracamy klasę o największej liczbie głosów
- też robi ensemble modeli, też działa dla dowolnych klasyfikatorów binarnych

OvR vs OvO - zalety i wady

- **szybkość i dodatkowy koszt:**
  - OvR trenuje mało modeli, ale każdy na pełnych danych
  - OvO trenuje dużo modeli, ale każdy na podzbiorze danych z 2 klas
  - OvR jest zwykle szybsze, ale dalej K razy droższe od klasyfikacji binarnej
  - OvO jest szybsze dla modeli, które mają dużą złożoność w liczbie próbek n, jak np. kernel SVM
  - OvO potrafi być bardzo szybkie dla dużej liczby klas (małe zbiory dla par klas)

OvR vs OvO - zalety i wady

- **prostota i interpretowalność:**
  - w OvR każda klasa ma "swój" klasyfikator, mocno do niej dostosowany
  - OvR jest prostsze i bardziej interpretowalne - można przeanalizować, czym różnią się modele dla poszczególnych klas
- **niezbalansowanie zbioru:**
  - OvR tworzy zbiory niezbalansowane - łączymy K-1 "przeciwnych" klas, więc właściwie zawsze będzie ich więcej niż klasy pozytywnej
  - OvO nic nie zmienia - klasy są tak zbalansowane, jak były pierwotnie

SVM a predykcje prawdopodobieństw

SVM a prawdopodobieństwa

- SVM potrafi przewidywać tylko klasyfikację binarną
- **obserwacja:** mamy wynik klasyfikatora (score), czyli jego pewność co do predykcji

$$f(x_i) = w^T x_i + b$$

- **idea:** przetłumaczmy jakoś tę pewność na prawdopodobieństwo

Platt scaling

- **obserwacja:** regresja logistyczna bardzo dobrze przewiduje prawdopodobieństwa - jest dobrze **skalibrowana (calibrated)**
- SVM zwraca wartość ciągłą
- połączenie tych dwóch idei to **Platt scaling** (od nazwiska twórcy - John Platt)
- **idea:** użyć przeskalowanej sigmoidy na wynikach z SVM

Platt, John. *"Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods."* Advances in large margin classifiers 10.3 (1999): 61-74.

Platt scaling

- **algorytm:**
  - bierzemy wynik z SVMa: $f(x_i) = w^T x_i + b$
  - wrzucamy go do 1-wymiarowej regresji logistycznej:

$$P(y = 1|x) = \frac{1}{1 + \exp(a * f(x) + b)}$$

  - zwracamy prawdopodobieństwo
- dalej potrafi klasyfikować - sprawdzamy, czy P(y=1|x) >= 0.5

Platt scaling

- **problem:** tę regresję logistyczną (współczynniki a i b) trzeba jeszcze wytrenować

$$P(y = 1|x) = \frac{1}{1 + \exp(a * f(x) + b)}$$

- używamy normalnie MLE, jak w regresji logistycznej
- **problem:** zbiorem treningowym są predykcje SVM, więc bardzo łatwo przetrenować
- **rozwiązanie:** użyć walidacji skrośnej
- Platt zaproponował też dodatkowe skalowanie zmiennych jako regularyzację

Platt scaling - zalety i wady

Zalety:

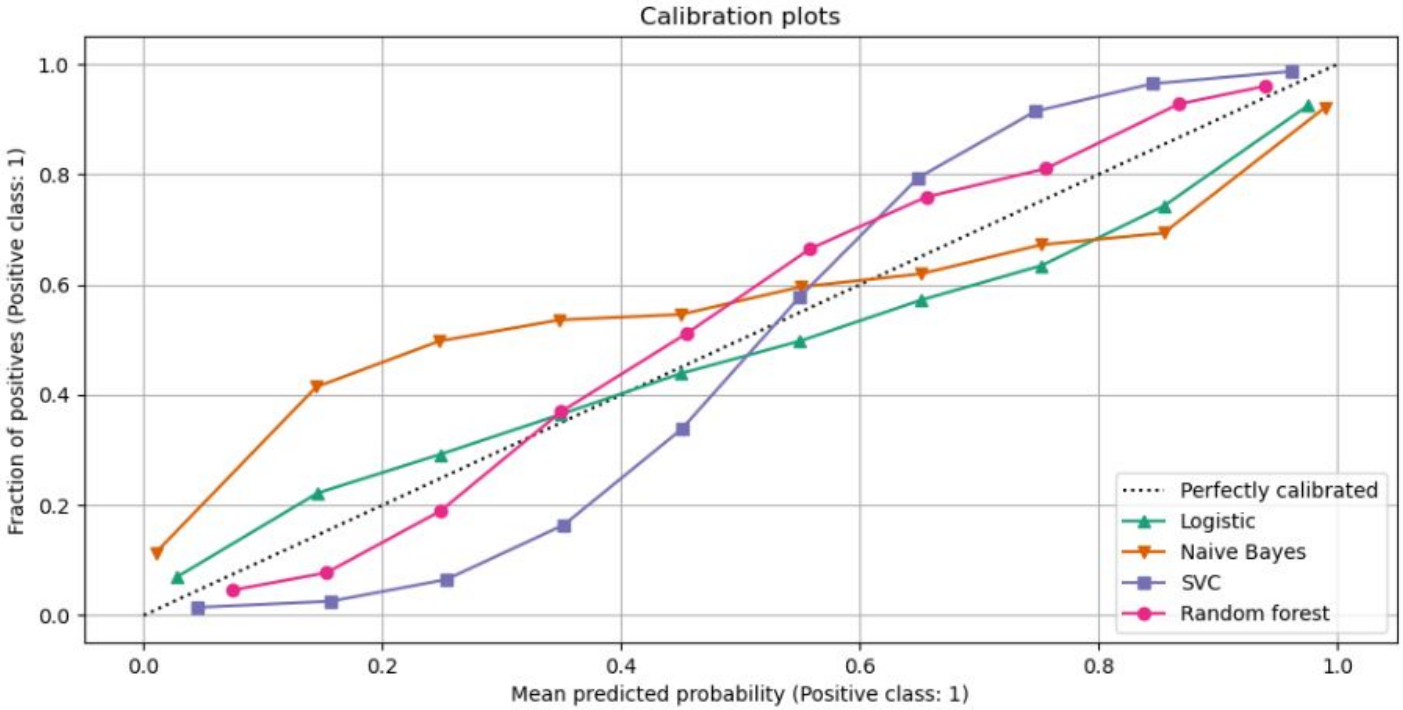
- pozwala na przewidywanie prawd. dla dowolnych klasyfikatorów binarnych
- prostota
- dość dobra kalibracja dzięki regresji logistycznej

Wady:

- duży koszt obliczeniowy przez walidację skrośną
- łatwo przeucza (ale w praktyce to nie duży problem)
- predykcje prawd. dalej nie są zbyt dobre

Platt scaling - kalibracja

Z typowych modeli SVM ma najgorszą kalibrację prawd.



Solwer PEGASOS

Solwer PEGASOS

- **Primal Estimated sub-GrAdient SOlver for SVM**

Shalev-Shwartz, S., Singer, Y., Srebro, N. et al. *"Pegasos: primal estimated sub-gradient solver for SVM"*. Math. Program. 127, 3–30 (2011).

- jeden z najprostszych i najefektywniejszych solwerów typu primal dla SVM
- zwykły SGD z subgradientem
- artykuł opisuje kilka wersji, my użyjemy najlepszej i najprostszej
- zakładamy, że bias jest dodany jako cecha stała x_0 = 1
- prezentacja MIT

Solwer PEGASOS

Epoka t = 1, 2, …, T:

1. Przemieszaj próbki (shuffle)
2. Ustaw learning rate $\eta_t = \frac{1}{\lambda t}$
3. Idź po kolei po przykładach, dla każdego:
   a. Oblicz predykcję $\hat{y}_i = w^T x_i$
   b. Jeżeli predykcja jest:
      i. prawidłowa ($y_i * \hat{y}_i \geq 1$), to: $w \leftarrow (1 - \eta_t \lambda) w$
      ii. nieprawidłowa ($y_i * \hat{y}_i < 1$), to: $w \leftarrow (1 - \eta_t \lambda) w + \eta_t y_i x_i$

Kernel primal SVM

- **da się** zrobić kernel primal SVM -> kernel PEGASOS
- stworzony dopiero w 2011 (oryginalny kernel SVM: 1995)
- kernelizacja praktycznie **nie zmienia** złożoności obliczeniowej!
- było znacznie ciężej to wymyślić

Solwer PEGASOS - zalety

Zalety:

- prostota
- szybka zbieżność
- skalowalność
- dobre wyniki
- mocne gwarancje teoretyczne
- udowodniony optymalny learning rate
- możliwość użycia kerneli

Kernel approximation

Kernel approximation

- **problem:** metody kernelwe mają zbyt duży koszt
- co najmniej raz używamy każdego elementu w macierzy kerneli - koszt to minimum O(n^2)
- **idea:** skoro aproksymacja zadziałała w kNN, to może zadziała też tutaj?
- dokładnie to robi **kernel approximation**

Kernel approximation

- obliczamy **cechy kernelowe (kernel features)**
- przybliżają one iloczyny skalarne między i-tą próbką a pozostałymi
- jest ich q, gdzie q << n
- q nie jest związane z oryginalną wymiarowością d
- te q wartości "agreguje" informacje z przestrzeni kernelowej
- skończona wymiarowość - im większa, tym dokładniejsza aproksymacja

Liczba cech w kernel approximation

- **więcej cech to niekoniecznie lepiej**
- im większa wymiarowość q, tym lepiej aproksymujemy kernel, ale niekoniecznie warto
- algorytm, którego potem używamy, może mieć problem ze zbyt dużą liczbą cech
- może nie być warto tak dokładnie aproksymować - mniejsza dokładność może służyć jako regularyzacja, np. ignorować szum w danych

Uniwersalność kernel approximation

- kernel approximation oddziela od siebie obliczanie cech i algorytm klasyfikacji
- approximate kernel SVM = kernel approximation features + linear SVM
- **obserwacja:** to zwykła transformacja cech, można użyć dowolnego algorytmu!
- działa dla dowolnych metod - nie tylko dla tych, które mają wersje kernelowe

Kernel approximation - algorytmy

- typowym algorytmem jest **metoda Nyströma (Nyström method):**
  - działa dla dowolnych kerneli
  - data-driven - próbkuje zbiór danych do uzyskania aproksymacji
  - typowo dość szybkie i daje bardzo dobre wyniki, bo mocno bazuje na danych
- dla poszczególnych kerneli istnieją bardziej wyspecjalizowane metody, które potrafią działać szybciej i/lub dokładniej:
  - RBF kernel: Random Fourier Features (RFF), Fastfood
  - polynomial kernel: TensorSketch

Metoda Nyströma

- najprostszy i najbardziej uniwersalny algorytm kernel approximation
- **algorytm:**
  - wybierz losowo q wierszy z macierzy X
  - przybliż macierz kerneli K za pomocą tych wierszy, otrzymując $\hat{K}_q$
  - oblicz rozkład SVD dla $\hat{K}_q$: $\hat{K}_q = \hat{U}_q \hat{\Sigma}_q \hat{V}_q^T$
  - **przybliżone cechy kernelowe:**
- żeby zrozumieć czemu, trzeba trochę wyprowadzeń i dowodów na macierzach
- wyprowadzenie (Wikipedia), alternatywne wyprowadzenie (CrossValidated)

Metoda Nyströma

- najprostszy i najbardziej uniwersalny algorytm kernel approximation
- **algorytm:**
  - wybierz losowo q wierszy z macierzy X
  - przybliż macierz kerneli K za pomocą tych wierszy, otrzymując $\hat{K}_q$
  - oblicz rozkład SVD dla $\hat{K}_q$: $\hat{K}_q = \hat{U}_q \hat{\Sigma}_q \hat{V}_q^T$
  - **przybliżone cechy kernelowe:** $\hat{X}_q = \hat{U}_q \hat{\Sigma}_q^{-\frac{1}{2}} \hat{V}_q^T$
- żeby zrozumieć czemu, trzeba trochę wyprowadzeń i dowodów na macierzach
- wyprowadzenie (Wikipedia), alternatywne wyprowadzenie (CrossValidated)

Metoda Nyström - złożoność obliczeniowa

- metoda sprowadza się do obliczenia macierzy kerneli na podzbiorze X i obliczeniu dla niej rozkładu SVD
- **złożoność obliczeniowa:** O(q^2 + q^3) = O(q^3)
- O(q^2) - macierz kernelowa, O(q * q^2) = O(q^3) - SVD

- **q to hiperparametr**, typowo w zakresie ok. 100-200
- w praktyce jest to bardzo szybkie i bardzo stabilne numerycznie dzięki SVD